

Wireless STM32 Flashing — How It Works, and Why It Took Two Days

This document explains how the spot welder flashes its **STM32G474** weld controller **wirelessly**, driven by the **ESP32-P4** (CrowPanel 5"), and gives an honest post-mortem of why getting it working took about two days.

It is written so that future-you (or anyone else) can re-derive the whole thing from scratch without re-living the dead ends.

1. The big picture

There are **two** processors on this board:

Chip	Role	Normal serial link
ESP32-P4	Touchscreen UI, WiFi, "the brain". Receives new firm-ware over WiFi.	—
STM32G474	Real-time weld controller ("the muscle").	UART @ 1,000,000 baud, 8N1

The goal: push a freshly built STM32 image **over WiFi to the ESP32**, and have the **ESP32 program the STM32** over the **two UART wires that already connect them** — no ST-Link, no USB cable, no taking the lid off.

This mirrors the old board's "espota + stm32flash" workflow, but the ESP32 now plays the role that a desktop PC + ST-Link used to play.

```
[ Your PC ] --WiFi--> [ ESP32-P4 ] --2 UART wires--> [ STM32G474 ]
stm32_flash_wifi.py   /stm32 endpoint          ROM bootloader
                      AN3155 flasher           (AN3155 protocol)
```

2. The wiring

Only the **two UART wires** are strictly required for flashing; the BOOT0/NRST pair is the hardware fallback (explained in §5).

ESP32-P4 GPIO	direction	STM32G474 pin	purpose
GPIO29 (TX)	→	PA10 (USART1_RX)	data ESP→STM
GPIO30 (RX)	←	PA9 (USART1_TX)	data STM→ESP
GPIO31	→	PB8 / BOOT0	hardware entry: force ROM at reset (10 kΩ pulldown on STM side)
GPIO32	→	NRST	hardware entry: pulse reset

The same two data wires carry **both** the normal 1 Mbaud application traffic **and** the 115200 8E1 bootloader traffic — the ESP just reconfigures its UART on the fly.

Board: WeAct STM32G474 Core Board V1.0. Note **VBAT is not wired**, which matters later (§6, the TAMP red herring).

3. The STM32 ROM bootloader (AN3155) in one paragraph

Every STM32 ships with a **factory ROM bootloader** burned in at `0x1FFF0000`. When entered, it speaks a simple UART protocol (ST app-note **AN3155**):

1. Host sends `0x7F` once to **auto-baud / sync**; ROM replies `0x79` (ACK).
2. Each command is a byte + its complement (e.g. Get-ID = `0x02 0xFD`), acknowledged with `0x79`.
3. Commands we use: **Get-ID** (returns `0x469` for G474), **Erase** (mass / page erase), **Write Memory** (`0x31`), **Read Memory** (`0x11`).
4. The bootloader UART runs at **8E1 (even parity)** — not the 8N1 the application uses. **This parity difference is critical** and was one of the silent failure modes.

So “flashing” = get into the ROM → sync → erase → write the `.bin` 256 bytes at a time → (now) read it back and verify → leave the ROM and run the app.

4. Getting INTO the ROM bootloader — the two paths

The whole problem reduces to one question: **how do you convince a running application to hand control to the ROM bootloader?** There are two ways, and we now support **both, independently selectable**, so each can be proven on its own.

Path A — Software entry (the wire-free dream, 2 wires only)

1. ESP sends the app a `BOOTLOADER` command over the normal 1 Mbaud link.

2. The STM32 app must then jump to the ROM **by itself** — no BOOT0, no NRST.

The naive way (“just set the vector table and jump from inside the app”) **does not work reliably** on the G4 and was the root of much pain (§6). The robust recipe we landed on:

1. App receives `BOOTLOADER`, writes a **magic marker into** `.noinit SRAM` (a RAM region the C startup does **not** wipe), and also into a TAMP backup register as a secondary fallback. → breadcrumb **stage 1**.
2. App calls `NVIC_SystemReset()` — a clean warm reset.
3. Very early in `main()`, **before** any peripheral init, the firmware checks the marker. If present, it clears it (one-shot) → breadcrumb **stage 2** → and calls `jumpToBootloader()`.
4. `jumpToBootloader()` tears everything down (`HAL_RCC_DeInit`, `HAL_DeInit`, disables the USB DP/DM pins PA11/PA12, sets `VTOR / MSP` to the ROM’s), → breadcrumb **stage 3** → and branches to `0x1FFF0000`.

Doing the jump **from a freshly-reset, near-bare machine** avoids the leftover interrupt/clock/USB state that makes a jump-from-running-app fault.

Status: this software path is wired up and instrumented but, on this specific board, the ROM still does not answer after the jump. The breadcrumb (below) tells us exactly how far it gets so we can finish diagnosing it.

Path B — Hardware entry (BOOT0 + NRST, the reliable one)

1. ESP drives **BOOT0 high** (GPIO31 → PB8).
2. ESP **pulses NRST low** (GPIO32) to reset the STM32.
3. The STM32 boots, sees BOOT0 high at reset, and enters the ROM bootloader directly. No cooperation from the app needed — works even on a half-bricked chip.

Status: this path **works end-to-end today** and is what actually flashes the chip right now.

Selecting a path — `?method=`

The `/stm32` HTTP endpoint accepts `?method=sw|hw|auto`:

value	behaviour
<code>auto</code> (default)	software jump first, hardware pulse as fall-back
<code>sw</code>	software 2-wire jump only (no BOOT0/NRST) — proves Path A in isolation
<code>hw</code>	BOOT0+NRST pulse only — proves Path B in isolation

From the flashing tool:

```
python tools/stm32_flash_wifi.py --method auto # default
python tools/stm32_flash_wifi.py --method sw  # software path only
python tools/stm32_flash_wifi.py --method hw  # hardware path only
```

5. Proving it actually worked — the read-back VERIFY pass

A subtle and embarrassing failure mode (§6) was a flash that **reported success but never actually programmed the chip**. To make success **undeniable**, after writing we now do a full **Read Memory (0x11) read-back** of every byte and compare against the source image:

```
VERIFY OK: all 65292 bytes
```

This is byte-for-byte. If even one byte differs, it fails loudly. Combined with an **unmistakable boot banner** the STM32 prints on every start-up:

```
BOOT,FW=N0INIT-MARKER-RESET-v4-WIRELESS-TEST
```

...we can now tell **with certainty** that the new image is the one running on the chip — not a stale build, not the old firmware.

6. Why it took two days — the honest post-mortem

None of the individual pieces are hard. What made it slow was a stack of **silent / misleading failures** that each looked like progress.

6.1 The killer: we were flashing nothing, for hours

Early wired flashes were **silently failing** — the mass-erase never actually happened, so the STM32 **kept running its OLD firmware the entire time**. Every STM32-side fix we made and “tested” was never actually on the chip. We were debugging code that wasn’t running. This is the single biggest reason for the lost time, and it’s exactly why §5 (read-back verify + boot banner) now exists: so this can **never silently happen again**.

6.2 The 8E1 vs 8N1 parity trap

The application link is **8N1**; the ROM bootloader is **8E1**. If the ESP listens with the wrong parity, a perfectly healthy reply from the STM32 is **silently dropped** by the UART driver and looks identical to “the chip is dead / silent”. We added UART **event-queue monitoring** specifically to catch parity/framing errors — that’s the difference between “it’s not responding” and “it’s responding but I’m mis-reading it”.

6.3 Red herrings that ate time

- **NRST “stuck low” false alarm:** the board has an RC cap on NRST; a naive read of the line looked like it was being held low when it wasn’t.
- **The `nBOOT_SEL` option-byte theory:** we spent time convinced the option bytes (`nBOOT_SEL=1`) were preventing BOOT0 from working. They weren’t the blocker.
- **False-positive “VERIFY OK”:** an early verify compared against a **stale buffer**, so it “passed” without proving anything. Now it reads back from the chip over the wire.
- **TAMP backup-register debug spam:** because VBAT isn’t wired, the TAMP backup domain clears to `0x00000000` on every reset. We initially read this as a bug. It’s expected on this board; the `.noinit` SRAM marker is the primary mechanism and TAMP is just a secondary fallback.

6.4 The broken software jump

The first software-entry implementation jumped to the ROM **from inside the running application’s context**. Leftover clock/interrupt/USB state made the core **fault and reset straight back into the app** — so it looked like the command “did nothing”. The fix was the marker+-warm-reset+-early-jump dance in §4 Path A.

6.5 The shared NRST/BOOT0 lines vs. the ST-Link (the wired-flash killer)

When you still flash/debug the STM32 over **SWD with an ST-Link**, the ST-Link **also drives NRST**. The ESP was holding **NRST and BOOT0 as strong push-pull outputs at all times** (max drive, NRST idle-HIGH). So two masters fought over the reset line:

- the ST-Link tried to pull NRST **low** to connect-under-reset, while
- the ESP shoved NRST **high** at ~40 mA.

The symptom is STM32CubeProgrammer’s **Error: ST-LINK error (DEV_TARGET_HELD_UNDER_RESET)** — and it only goes away when you **disconnect the ESP**. Worse, this same contention can corrupt an SWD flash mid-program: a GDB/CubeProgrammer **verify shows some sections “matched” but e.g. `.data` “MIS-MATCHED!”**, because the ST-Link couldn’t cleanly reset the target while the ESP was fighting it. That partial-write looks like a mysterious “one section won’t take” bug but is really a **two-drivers-on-one-line hardware conflict**.

The fix: the ESP now keeps **both NRST and BOOT0 high-impedance (Hi-Z input) whenever it is not mid-flash**, and only briefly drives them during its own hardware-entry / launch pulses (returning them to Hi-Z immediately after). The board’s own pulls give the right idle state with the ESP off the lines:

- BOOT0 (PB8): 10 kΩ board pulldown → floats LOW → app mode
- NRST: STM32 internal ~40 kΩ pull-up → floats HIGH → not in reset

With the lines released, the **ST-Link and the ESP can stay wired up at the same time** — no more `DEV_TARGET_HELD_UNDER_RESET`, and no more half-written SWD flashes.

6.6 Remote debugging latency

The assistant builds the STM32 firmware in a Linux VM (PlatformIO), but the **user builds and flashes on Windows** and reports back the serial logs. Every hypothesis required a round-trip: change → user builds → user flashes → user pastes 1 Mbaud serial → analyse. That loop is inherently slow, and it's why the diagnostics below were added — to get the **maximum information per round-trip**.

7. The breadcrumb — finishing the software path

To diagnose why the **software** path's ROM stays silent without endless round-trips, the STM32 now drops a **breadcrumb** in `.noinit` SRAM at each step of the jump. `.noinit` survives **every** reset except a true power-on (so it survives both the warm `NVIC_SystemReset()` and the hardware NRST pulse), and the value is tagged with a magic (`0x57130000`) so power-on garbage can't be mistaken for a real breadcrumb.

stage stamped	meaning
1	B00TL0ADER cmd received, marker set, about to <code>NVIC_SystemReset()</code>
2	after reset, early- <code>main()</code> saw the marker, about to <code>jumpToBootloader()</code>
3	inside <code>jumpToBootloader()</code> , about to branch into the ROM at <code>0x1FFF0000</code>

On the next boot the firmware prints, once:

```
DBG,BL breadcrumb: last software ROM-entry reached stage N/3
```

Reading the result:

- **stage 3** → we reached the ROM jump; any remaining failure is on the **ROM/USART side** (e.g. parity, or the ROM genuinely didn't take over).
- **stage 2** → the jump **faulted before branching** (teardown sequence issue).
- **stage 1** → the marker was **lost across the reset** (early jump skipped).

How to use it

1. Flash with the software path only:

```
python tools/stm32_flash_wifi.py --method sw
```
2. Capture the **STM32** serial @ **1,000,000 baud 8N1** across the reboot.
 (The ESP log is not enough — we need the STM32's own output to see the breadcrumb and the boot banner.)
3. Report the `DBG,BL breadcrumb: ... stage N/3` line. That single line tells us exactly which sub-problem to fix next.

Stage 3 revealed the answer → the BOOT0 fix

Field testing showed **stage 3/3** — the jump to ROM executed successfully, but the ROM stayed silent on USART1. The ESP received bytes (0x00 , 0x1B) but not the 0x79 ACK.

Root cause: the ROM bootloader **checks BOOT0 when it starts**, even if we jumped to it via VTOR/MSP. If BOOT0 is LOW the ROM either:

- exits back to app mode immediately (thinking “no bootloader requested”), OR
- waits on USB/I2C instead of USART1 (based on pin sampling per AN2606).

The fix: before the warm reset in `requestBootloaderReset()`, the STM32 app now **drives PB8 (BOOT0) HIGH** via GPIO push-pull output. The 10 kΩ board pulldown on PB8 is easily overcome by a strong (20 mA+) GPIO drive. When the ROM starts it samples BOOT0=HIGH → “bootloader mode requested” → stays in ROM and responds on USART1 @ 115200 8E1.

This fix makes the **software 2-wire path fully functional** — no ESP BOOT0/NRST wires needed, the STM32 drives its own BOOT0 internally before the reset.

8. Quick reference

```
# Build STM32 firmware (PlatformIO)
cd STM32G474CE && pio run -e g474ceu6_stlink

# Flash wirelessly through the ESP32
cd ESP32P4
python tools/stm32_flash_wifi.py           # AUTO (sw then hw fallback)
python tools/stm32_flash_wifi.py --method sw # software 2-wire path only
python tools/stm32_flash_wifi.py --method hw # hardware BOOT0+NRST path only
python tools/stm32_flash_wifi.py 192.168.1.77 --method hw # explicit host
```

Success looks like (ESP side):

```
sync OK (ACK)
product ID = 0x469
... write ...
VERIFY OK: all NNNNN bytes
```

...and (STM32 side, @1Mbaud) the new boot banner:

```
BOOT,FW=NOINIT-MARKER-RESET-v4-WIRELESS-TEST
```

thing	value
App UART	1,000,000 baud, 8N1
ROM bootloader UART	115200 baud, 8E1
STM32 product ID	0x469
ROM bootloader address	0x1FFF0000
App flash base	0x08000000
.noinit breadcrumb	g_bl_stage @ 0x2000cea0
.noinit boot marker	g_bootloader_marker @ 0x2000cea4
Marker magic	0x57130000 (breadcrumb)